

Stack Resource Policy and Scheduling Analysis

D7020E Robust and Energy Efficient Real-Time Systems 7

Pawel Dzialo

1.12.2025

Prof. Per Lindgren

L
LULEÅ
TEKNISKA
UNIVERSITETEN

L
LULEÅ
UNIVERSITY
OF TECHNOLOGY

Correctness under Real-Time Systems

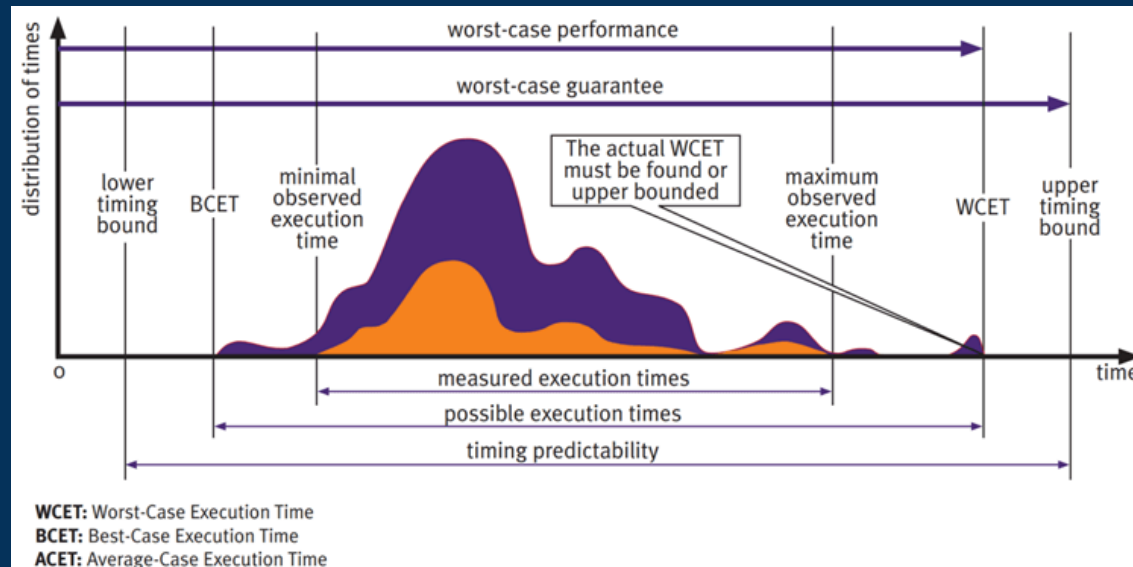
- A real-time system is a system working under some sort of **deadline**
 - In a **soft** real-time system, missing a deadline might cause service *degradation*
 - I.e. a video player should process each frame within the frame period
 - Say 24 Frames Per Second, $\frac{1s}{24} \approx 41ms$ for processing each frame before the video starts to stutter
 - Stutter degrades the experience, but it's recoverable and not catastrophic
 - Modern video formats are based on deltas from a keyframe, dropping one of those is more of an issue, but still recoverable unless we are consistently missing keyframes
 - Under **hard** real-time, the deadline is **fundamental** to the functionality
 - In a control loop, the controller is designed under an assumed loop frequency, missing this can make the plant explode past the point of controllability
 - More clearly, missing a deadline on actuating the brakes on a car might make them useless

Program Correctness

- In the previous lecture, we talked about ways of ensuring *total correctness* as defined in field of program verification
 - Liveness
 - Safety
- In a hard real-time system, missing a deadline might be behaviorally equivalent to producing the wrong result, or not halting at all
 - It doesn't matter if brakes fail due to incorrect actuation, no actuation at all, or actuation, but in 2 hours, one ends up in the same accident in all of these cases
- To be able to make any sort of real-world safety claims about our hard real-time system, we need to be able to reason around some form of **timeliness**, or adherence to deadlines
 - Timeliness is a stricter variation of liveness, we want the piece of code to not only halt, but to halt within some given time frame

Worst-Case Execution Time(WCET)

- The worst-case execution time(WCET) of a piece of code, is the longest possible time the piece of code could take to halt
- In industry, the approach to WCET is commonly similar to the approach to verification
 - Manual testing, fuzzing, proven-in-use
 - Amazingly, this (mostly) works (until it doesn't), is this a sound approach?



Worst-Case Execution Time(WCET)

- In Lab 3, you used Symex
 - One of the examples, you used Symex to obtain a **safe** upper bound on the execution time of a function
 - Safe here means the function is **guaranteed** to finish within the obtained bound (barring bugs in the tool itself)
 - We explore **all** possible paths through the function
 - We do this on machine-code level, each instruction has an associated worst-case execution time provided by the vendor
 - The upper bound WCET for the function is obtained by tallying up the WCET for each instruction encountered along each path, and choosing the path with the longest WCET tally
 - Already here you are miles ahead of most of industry, *hard* WCET analysis is applied in nuclear, avionics, small parts of automotive, that's pretty much it

WCET and Scheduling Analysis

- Embedded systems often have multiple distinct jobs to perform
 - Data acquisition and processing from sensors, reacting to input by e.g. writing to actuator, logging, communications, OTA updates, etc.
- Seemingly simplest way of handling this is by **parallelism**
 - One code execution unit (i.e. core) per job
 - This is not a luxury we have in resource constrained embedded systems
 - nRF 52840 is single-core, you may run into e.g. dual, triple, quad core embedded processors but not much more
 - Realistic systems consist of 10's-100's of distinct tasks, we cannot just keep adding cores
 - Even if we could, systems are never completely **embarrassingly parallel**
 - In reality, one can rarely avoid a certain amount of inter-job communication
 - E.g. data acquisition job must be able to communicate the data to the processing job
 - Communication means shared state means **synchronization** (e.g. Mutex) is required
 - A job may end up having to wait until a lock is released before it can continue
 - This means, the WCET of a function representing a job is *part* of the equation, rarely the full story

Non-preemptive scheduling

- A Job J_i is a *finite* sequence of instructions
 - E.g. a terminating function, an interrupt handler
- A Job Request $\mathcal{J}_i$ is one request for the execution of J_i
 - E.g. an interrupt signal going high
- The time between arrival of $\mathcal{J}_i$ and termination of J_i is R_i , or the response time of J_i
- Simplest way of scheduling multiple jobs in a single-core context is **non-preemptive** scheduling
- On $\mathcal{J}_a$, jump to J_a and execute until termination
 - If J_b is already executing, instead insert $\mathcal{J}_a$ into a queue, once J_b finishes, pop the topmost $\mathcal{J}_i$ from the queue and dispatch it
 - Queue could be e.g. FIFO, if we have a notion of job priorities relative each other could also be priority queue

Non-preemptive scheduling

- Simple and naive approach to scheduling
 - Real-time jobs have relative importance
 - E.g. logging is a cherry on top, never the core function of a system
 - If a logging job has been dispatched, any higher importance job must wait until logging is finished
- *Blocking time* B_i of J_i is the worst-case time a request J_i ends up waiting for lower priority jobs
 - Defining p_i as the *static* priority of J_i and C_i as its WCET
 - Assuming the job queue is a priority queue
 - $B_i = \max(\{C_n \mid p_n < p_i\})$, longest WCET of all lower priority jobs
- This is a simplified view, we are ignoring e.g. costs of enqueueing arriving job requests
 - Point is this is not the way to go about this



Preemptive scheduling

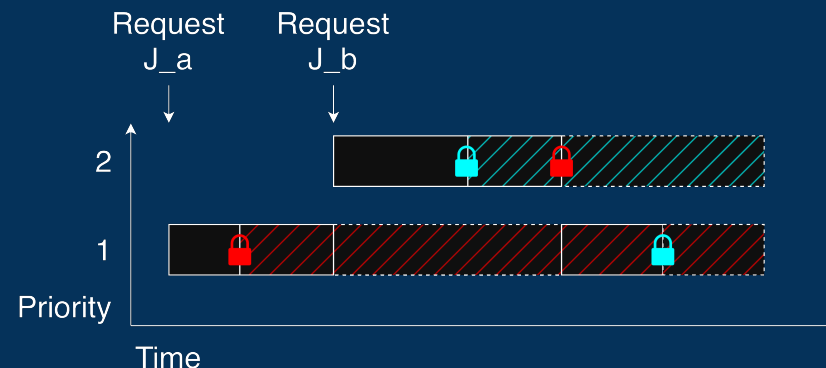
- Preemptive scheduling allows one job to be interrupted or **preempted** by another job
 - Under which conditions depends on the applied scheduling policy
- Most commonly implemented by **timeslicing**
 - Each J_i is assigned a set amount of time (a **quanta**) Q_i for which it gets to execute
 - If J_i doesn't terminate within Q_i it's interrupted and another job gets to execute for its quanta
 - Usually, jobs are scheduled in a **round-robin** fashion
 - Consider J_1, J_2, J_3 , each running for Q_1, Q_2, Q_3
 - J_1 doesn't finish within Q_1 , is interrupted, and J_2 gets to run for Q_2
 - J_2 doesn't finish either, Q_2 expires, and J_3 is dispatched
 - J_3 finishes ahead of time, next outstanding request is J_1 , we go back to executing it, and so on

Round-robin Scheduling in Real-Time Systems

- How can we apply timeslicing to real-time systems?
 - We have not yet addressed priorities, we may e.g. run one time-slicing instance per priority level
 - We round-robin only jobs of priority p until all have terminated, only then do we handle jobs $p - 1$
 - We have also not addressed resource sharing
 - A resource shared between two (or more) jobs in a preemptive context must always be **locked**
 - The aim of locking is to prevent simultaneous access to the resource by separate jobs, i.e. a race condition
 - Recall pointer aliasing? This is very similar
 - Under non-preemptive scheduling, this was handled by the nature of the scheme, each job runs to completion, so it will be done with a resource by the time we do something else

Resource sharing under Round-robin

- To address resource sharing we may implement some form of mutual exclusion
 - Familiar concept to you, refresher:
 - Each shared resource S_i is only accessible when holding a Mutex M_i over it
 - M_i may only be taken once at a time. Any attempts by J_a to take M_i while it is being held by e.g. J_b results in J_a entering a **blocked** state.
 - J_a is blocked, i.e. stopped from executing until J_b has released M_i
 - Mutexes are a common way of handling shared resources, however require care due to potential for **deadlocking**



Round-robin Scheduling in Real-Time Systems

- Round-robin scheduling itself may work but is expensive
 - Potentially multiple context switches per job request, each of these has a cost
 - Notice, a one cycle difference in WCET of a function, or length of quanta may add an entire timeslice to the response time of the job,
- Mutexes can deadlock
 - How to reason about response time of a system which may not be live?
 - One really doesn't...

Round-robin Scheduling in Real-Time Systems

- Regardless, state-of-the-art real-time operating systems, e.g. FreeRTOS(Amazon), Zephyr(Linux Foundation) implement time-slicing with shared resources through mutexes
 - Why? Your guess is as good as mine
 - Perhaps like in the C/C++ case, lack of **familiar** alternatives
- From FreeRTOS book: “the best method of avoiding deadlock is to consider its potential at design time, and design the system to ensure that deadlock cannot occur.”
 - Much like MISRA C rule 1.3: “There shall be no occurrence of undefined ... behaviour”
 - In other words: get good :)
- Opinion: in case of FreeRTOS or Zephyr, real-time is a buzzword, it doesn't mean anything

Alternative Resource Access Protocols

- **Stack Resource Policy(SRP)**, introduced by Ted Baker in 1990, is a Resource Access Protocol
 - A scheme for safe (e.g. race-free) sharing of resources between concurrent jobs
- Under SRP, certain properties of the system are **proven**
 - No race conditions
 - No deadlocks
 - Bounded (and easily deriveable) priority inversion
 - One context switch per job request (recall, this is not the case for time slicing)
 - Given our system, i.e. set of tasks represents a valid SRP model, all of these properties are transitively **proven** for our system
- Let's have a look at how these guarantees are achieved

SRP definition

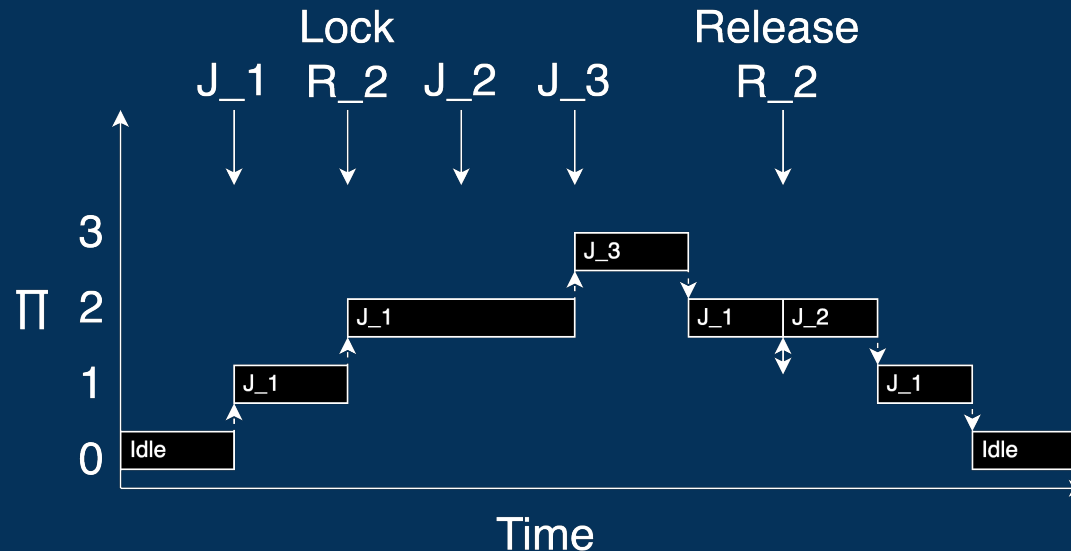
- As previously, job J_i is a **finite** sequence of instructions, $\mathcal{J}_i$ is a request for it's execution
- The priority P_i of J_i is the relative importance of J_i , $P_a > P_b$ means J_a is more important
- A shared resource S_k may be accessed by J_i by executing a critical section $Z_{k,i}$
 - The set of tasks with access to S_k is $L(S_k)$
 - Critical sections may be **nested**, i.e. we may be within more than one critical section at once
 - Nested critical sections **must** be entered/exited in a LIFO order, i.e. entering $Z_{k,i}$ then $Z_{l,i}$ means we must exit $Z_{k,i}$ before exiting $Z_{l,i}$, this prevents deadlocks
- Each shared resource S_k is associated with a **priority ceiling** $\lceil S_k \rceil$
 - $\lceil S_k \rceil = \max(\{0\} \cup \{P_i | J_i \in L(S_k)\})$, in other words $\lceil S_k \rceil$ is the largest priority of tasks with access to S_k

Job	Priority	Accessed Resources
J_1	1	R_1, R_2, R_3
J_2	2	R_2
J_3	3	R_3

Resource	Priority Ceiling
R_1	1
R_2	2
R_3	3

SRP definition

- The *current system ceiling* $\Pi = \max(\{0\} \cup \{P_i \mid J_i \in J_c\} \cup \{\lceil S \rceil \mid S \in S_c\})$, where J_c, S_c are the sets of currently executing jobs, and currently locked resources respectively
 - In other words, the system ceiling is the largest priority across all currently executing jobs, and the ceilings of all currently held shared resources
- A job request J_i is dispatched **iff** $P_i > \Pi$



SRP in practice

- If following the SRP model, we get guarantees to:
 - Deadlock freedom
 - Bounded priority inversion (more on this in short while)
 - One-and-only-one context switch per job request (highly efficient compared to round-robin)
- But, how do we ensure that our system is a valid SRP model? Are we back to get-good-land?
- If you've done the labs, you've implemented valid SRP models
 - RTIC **guarantees** (barring unsafe) that any compiling application is valid under SRP
 - All SRP guarantees also apply to **all** compiling RTIC applications

SRP in practice

- Originally, SRP was defined under Earliest Deadline First(**EDF**) *scheduling algorithm*
 - Earliest Deadline First assigns a deadline D_i to each job J_i
 - As J_i arrives, an absolute deadline (current time + D_i) for the request is calculated
 - Requests are dispatched according to the shortest absolute deadline
 - Priorities of jobs relative each other are dynamic
 - If J_a with a large D_a has waited for long enough, it will be dispatched ahead of J_b with a small D_b
- In practice, this means we need to maintain a queue of job requests and their relative absolute deadlines
- Each job request must be inserted into the queue as it arrives, i.e. the scheduler preempts the currently running job
- In the general case, this must be done in software, i.e. the cost of the scheduler itself is implementation dependent

SRP in practice

- RTIC implements SRP under **static priorities** instead of EDF
 - Each job is assigned a priority by the programmer at compile time
 - These priorities never change during runtime
- This allows us to map scheduling directly to the interrupt controller
 - Each job is mapped to an interrupt
 - II is implemented through the NVIC BASEPRI register
 - Locking S_i = setting BASEPRI to $\lceil S_i \rceil$ (single cycle operation)
 - Unlocking S_i = restoring BASEPRI to original value (single cycle operation)
 - Dispatching requests done in hardware, entirely for free
 - Regardless of the actual cost of operation, they are constant time, which is nice when reasoning around worst-case overhead of the scheduler
 - Of course, the cost being 0-1 processor cycles is also nice, and blows anything else completely out of the water

Scheduling analysis under SRP

- Aside from static guarantees to e.g. deadlock freedom, SRP is also amenable to **scheduling analysis**
 - What is the **response time** R_i of a job? I.e. within how much time is J_i **guaranteed** to be handled?
 - Is the system as a whole **schedulable**? I.e. does our core run quickly enough to satisfy all potential request deadlines in perpetuity?
- Starting with response time, generally defined as a sum of three numbers
 - B_i , the blocking time of J_i , worst-case time J_i might spend waiting for **lower priority** jobs to e.g. release a resource, this is essentially unavoidable priority inversion
 - I_i , the interference of J_i , worst-case time J_i might spend waiting for **higher or equal priority** jobs to finish executing
 - C_i , the worst-case execution time of J_i
 - $R_i = B_i + I_i + C_i$, this should make sense intuitively, what else could possibly affect R_i ?

Scheduling analysis under SRP

- $R_i = B_i + I_i + C_i$
 - C_i is just the worst-case execution time, can be approximated by measurements, or a safe bound can be obtained using e.g. Symex
 - B_i , when is J_i ever blocked?
 - Whenever J_a is holding R_a where $\lceil R_a \rceil > p_i$
 - When R_i is released, J_i immediately gets to preempt
 - As such B_i is the longest critical section over a resource with higher priority ceiling than the priority of J_i , executed by a job with priority lower than J_i
 - Formally, we build the set of critical sections belonging to tasks of priority lower than P_i , over resources with $\lceil S_k \rceil \geq P_i$, $\gamma_i = \{Z_{j,k} \mid P_j < P_i \wedge P_i \leq \lceil S_k \rceil\}$
 - If $\delta_{j,k}$ is the WCET of $Z_{j,k}$, $B_i = \max(\{\delta_{j,k} \mid Z_{j,k} \in \gamma_i\})$
 - The WCET of critical sections can again be approximated by measurement, or bounded by Symex

Scheduling analysis under SRP: Preemption overapproximation

- $R_i = B_i + I_i + C_i$
 - We've covered B_i , C_i , what about I_i ?
 - Here, the situation gets slightly hairier
- How many times can a specific request $\mathcal{J}_i$ be preempted?
 - An overapproximation
 - The busy-time Bp_i of J_i is the worst-case time between arrival of $\mathcal{J}_i$ and it's termination
 - Notice, this is essentially the response time of J_i , for now we are working with an overapproximation, we will address this later
 - Assuming system is schedulable, Bp_i can never be longer than the deadline D_i of J_i
 - The deadline is a design parameter, for instance control loop frequency
 - $Bp_i > D_i$ means we start missing deadlines
 - For our overapproximation, let's assume $Bp_i = D_i$

Scheduling analysis under SRP: Preemption overapproximation

- J_i is associated with an interarrival period A_i
 - The smallest possible time between arriving execution requests for J_i
 - E.g. if polling sensors at 100Hz, A_i for a processing job would be $\frac{1}{100}$ s
- Intuitively, we are after the preemptions during the busy-period Bp_i of J_i
 - J_i may only be preempted by J_a if $P_a > J_i$
 - Requests J_a are assumed to arrive with period A_a in the worst case
 - Hence, during Bp_i , we may dispatch and handle $\left\lceil \frac{Bp_i}{A_a} \right\rceil$ instances of J_a
 - We will spend $C_a * \left\lceil \frac{Bp_i}{A_a} \right\rceil$ handling preemptions cause by J_a
 - This applies to all jobs with a priority higher than J_i
 - I.e. $I_i = \sum_{n:P_n > P_i} C_n * \left\lceil \frac{Bp_i}{A_n} \right\rceil$

Preemption overapproximation worked example

Job	P	Deadline	WCET	Interarrival	Response Time
J_1	1	56ms	10ms	100ms	?
J_2	2	20ms	6ms	70ms	?
J_3	3	10ms	2ms	12ms	?

CS	Resource	WCET	In Job
$Z_{1,1}$	S_1	2ms	J_1
$Z_{2,1}$	S_1	4ms	J_2

- $R_3 = B_3 + I_3 + C_3$
 - S_1 is the only resource, with $\lceil S_1 \rceil < P_3$, hence J_3 cannot be blocked, $B_3 = 0\text{ms}$
 - J_3 has the largest priority, hence it cannot be preempted, $I_3 = 0\text{ms}$
 - C_3 is visible directly in the table, 2ms
- $R_3 = B_3 + I_3 + C_3 = 0\text{ms} + 0\text{ms} + 2\text{ms} = 2\text{ms}$

Preemption overapproximation worked example

Job	P	Deadline	WCET	Interarrival	Response Time
J_1	1	56ms	10ms	100ms	?
J_2	2	20ms	6ms	70ms	?
J_3	3	10ms	2ms	12ms	2ms

CS	Resource	WCET	In Job
$Z_{1,1}$	S_1	2ms	J_1
$Z_{2,1}$	S_1	4ms	J_2

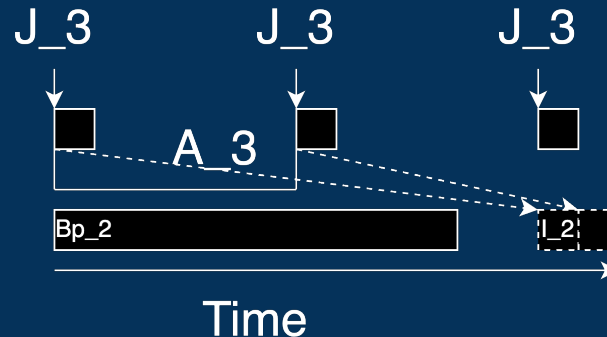
- $R_2 = B_2 + I_2 + C_2$
 - S_1 has $\lceil S_1 \rceil \geq P_2$
 - $B_2 = \max(\{\delta_{j,k} \mid Z_{j,k} \in \gamma_2\})$, where $\gamma_2 = \{Z_{j,k} \mid P_j < P_2 \wedge P_2 \leq \lceil S_k \rceil\}$
 - In other words γ_2 is set of all critical sections belonging to jobs of lower priority than J_2 , over resources with higher or equal priority ceiling to P_2 in this case just $\{Z_{1,1}\}$.
 - $\delta_{1,1} = 2\text{ms}$ is visible from the table. $B_2 = \max(\{2\text{ms}\}) = 2\text{ms}$

Preemption overapproximation worked example

Job	P	Deadline	WCET	Interarrival	Response Time
J_1	1	56ms	10ms	100ms	?
J_2	2	20ms	6ms	70ms	?
J_3	3	10ms	2ms	12ms	2ms

CS	Resource	WCET	In Job
$Z_{1,1}$	S_1	2ms	J_1
$Z_{2,1}$	S_1	4ms	J_2

- $R_2 = B_2 + I_2 + C_2 = 2\text{ms} + I_2 + C_2$
 - J_2 can be preempted by J_3 since $P_3 > P_2$
 - We assume worst-case $Bp_2 = D_2 = 20\text{ms}$
 - $I_2 = \sum_{n:P_n > P_2} C_n * \left\lceil \frac{Bp_2}{A_n} \right\rceil = 2\text{ms} * \left\lceil \frac{20\text{ms}}{12\text{ms}} \right\rceil = 2\text{ms} * 2 = 4\text{ms}$



Preemption overapproximation worked example

Job	P	Deadline	WCET	Interarrival	Response Time
J_1	1	56ms	10ms	100ms	?
J_2	2	20ms	6ms	70ms	?
J_3	3	10ms	2ms	12ms	2ms

CS	Resource	WCET	In Job
$Z_{1,1}$	S_1	2ms	J_1
$Z_{2,1}$	S_1	4ms	J_2

- $R_2 = B_2 + I_2 + C_2 = 2\text{ms} + 4\text{ms} + C_2$
 - $C_2 = 6\text{ms}$ is already in the table
 - $R_2 = B_2 + I_2 + C_2 = 2\text{ms} + 4\text{ms} + 6\text{ms} = 12\text{ms}$

Preemption overapproximation worked example

Job	P	Deadline	WCET	Interarrival	Response Time
J_1	1	56ms	10ms	100ms	?
J_2	2	20ms	6ms	70ms	12ms
J_3	3	10ms	2ms	12ms	2ms

CS	Resource	WCET	In Job
$Z_{1,1}$	S_1	2ms	J_1
$Z_{2,1}$	S_1	4ms	J_2

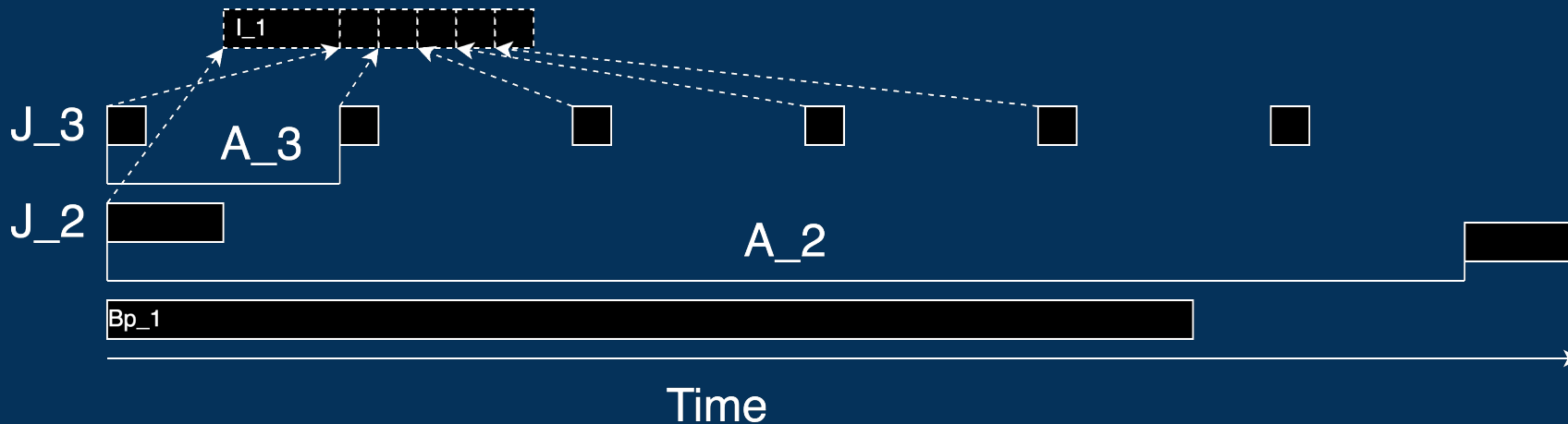
- $R_1 = B_1 + I_1 + C_1$
 - J_1 is of lowest priority, it cannot be blocked, $B_1 = 0\text{ms}$
 - C_1 is, again, already in the table, $C_1 = 10\text{ms}$
 - What about I_1 ? J_1 can be preempted by both J_2 and J_3
 - $I_1 = \sum_{n:P_n > P_1} C_n * \left\lceil \frac{Bp_1}{A_n} \right\rceil = C_2 * \left\lceil \frac{Bp_1}{A_2} \right\rceil + C_3 * \left\lceil \frac{Bp_1}{A_3} \right\rceil$

Preemption overapproximation worked example

Job	P	Deadline	WCET	Interarrival	Response Time
J_1	1	56ms	10ms	100ms	?
J_2	2	20ms	6ms	70ms	12ms
J_3	3	10ms	2ms	12ms	2ms

CS	Resource	WCET	In Job
$Z_{1,1}$	S_1	2ms	J_1
$Z_{2,1}$	S_1	4ms	J_2

- $R_1 = B_1 + I_1 + C_1 = 0\text{ms} + I_1 + 10\text{ms}$
 - $I_1 = \sum_{n:P_n > P_1} C_n * \left\lceil \frac{Bp_1}{A_n} \right\rceil = C_2 * \left\lceil \frac{Bp_1}{A_2} \right\rceil + C_3 * \left\lceil \frac{Bp_1}{A_3} \right\rceil = 6\text{ms} * 1 + 2\text{ms} * 5 = 16\text{ms}$



Preemption overapproximation worked example

Job	P	Deadline	WCET	Interarrival	Response Time
J_1	1	56ms	10ms	100ms	22ms
J_2	2	20ms	6ms	70ms	12ms
J_3	3	10ms	2ms	12ms	2ms

CS	Resource	WCET	In Job
$Z_{1,1}$	S_1	2ms	J_1
$Z_{2,1}$	S_1	4ms	J_2

- $R_1 = B_1 + I_1 + C_1 = 0\text{ms} + 12\text{ms} + 10\text{ms} = 22\text{ms}$

Scheduling analysis under SRP: Precise preemption

- Recall, the busy period Bp_i is really the amount of time between request J_i arrives, and the exit of J_i , this is our response time!
 - In the base case, where no preemptions take place, the response time is $R_i = B_i + C_i$
 - How many preemptions can take place under the base case response time?
 - The same question as previously, the answer is $\left\lceil \frac{R_i}{A_n} \right\rceil = \left\lceil \frac{B_i + C_i}{A_n} \right\rceil$, for each job with priority higher than P_i
 - The total preempted time becomes $I_i = \sum_{n:P_n > P_i} C_n * \left\lceil \frac{B_i + C_i}{A_n} \right\rceil$, and the new response time $R_i = B_i + C_i + \sum_{n:P_n > P_i} C_n * \left\lceil \frac{R_i}{A_n} \right\rceil$
 - Notice, the definition of response time becomes recursive, we express this as a recurrence relation:

$$R_i := \begin{cases} R_i^{(0)} = B_i + C_i \\ R_i^{(s)} = B_i + C_i + \sum_{n:P_n > P_i} C_n * \left\lceil \frac{R_i^{(s-1)}}{A_n} \right\rceil \end{cases}$$

Scheduling analysis under SRP: Precise preemption

- We can obtain a final response time value for J_i by iterating over the recurrence relation until it converges, i.e. $R_i^{(s)} = R_i^{(s-1)}$
 - Notice, in cases where J_i is generally unschedulable, the relation may not converge
 - In that case, it's reasonable to stop once $R_i > D_i$, at this point we are breaking deadlines, and need to go back to the drawing board anyways

Precise preemption worked example

Job	P	Deadline	WCET	Interarrival	Response Time	B_i
J_1	1	56ms	10ms	100ms	?	0ms
J_2	2	20ms	6ms	70ms	?	2ms
J_3	3	10ms	2ms	12ms	2ms	0ms

CS	Resource	WCET	In Job
$Z_{1,1}$	S_1	2ms	J_1
$Z_{2,1}$	S_1	4ms	J_2

- Going back to the previous example, only I_i has changed, we can keep the blocking time we obtained previously
- We can also notice that J_3 still cannot be preempted, it's the highest priority
 - Hence, $I_3 = 0\text{ms}$ still applies, $R_3 = B_3 + I_3 + C_3 = 0\text{ms} + 0\text{ms} + 2\text{ms} = 2\text{ms}$, same as before

Precise preemption worked example

Job	P	Deadline	WCET	Interarrival	Response Time	B_i
J_1	1	56ms	10ms	100ms	?	0ms
J_2	2	20ms	6ms	70ms	?	2ms
J_3	3	10ms	2ms	12ms	2ms	0ms

CS	Resource	WCET	In Job
$Z_{1,1}$	S_1	2ms	J_1
$Z_{2,1}$	S_1	4ms	J_2

- What about R_2 ?
 - Again, blocking time is not affected by size of the busy period, it's only a measure of how long we may have to wait at request arrival
 - In other words, blocking time is same as before, we can reuse the results
 - C_2 is still just the WCET of J_2 , and can be obtained from the table
 - Recall recurrence relation

$$R_2 := \begin{cases} R_2^{(0)} = B_2 + C_2 = 2\text{ms} + 6\text{ms} = 8\text{ms} \\ R_2^{(s)} = B_2 + C_2 + \sum_{n:P_n > P_2} C_n * \left\lceil \frac{R_2^{(s-1)}}{A_n} \right\rceil = 8\text{ms} + \sum_{n:P_n > P_2} C_n * \left\lceil \frac{R_2^{(s-1)}}{A_n} \right\rceil \end{cases}$$

Precise preemption worked example

Job	P	Deadline	WCET	Interarrival	Response Time	B_i
J_1	1	56ms	10ms	100ms	?	0ms
J_2	2	20ms	6ms	70ms	?	2ms
J_3	3	10ms	2ms	12ms	2ms	0ms

CS	Resource	WCET	In Job
$Z_{1,1}$	S_1	2ms	J_1
$Z_{2,1}$	S_1	4ms	J_2

$$R_2 := \begin{cases} R_2^{(0)} = B_2 + C_2 = 2\text{ms} + 6\text{ms} = 8\text{ms} \\ R_2^{(s)} = B_2 + C_2 + \sum_{n:P_n > P_2} C_n * \left\lceil \frac{R_2^{(s-1)}}{A_n} \right\rceil = 8\text{ms} + \sum_{n:P_n > P_2} C_n * \left\lceil \frac{R_2^{(s-1)}}{A_n} \right\rceil \end{cases}$$

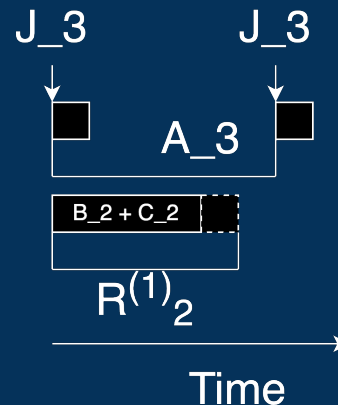
- Intuitively, the base case is the minimum amount of time we will spend waiting for J_2
 - We calculate how many preemptions may happen within this time
 - When waiting for J_2 , we will also have to wait for any preemptions, i.e. the preemptions extend the waiting period
 - Given the new waiting period, we recount the amount of preemptions that may happen, and we keep doing this until the amount of preemptions stabilizes (or doesn't)

Precise preemption worked example

Job	P	Deadline	WCET	Interarrival	Response Time	B_i
J_1	1	56ms	10ms	100ms	?	0ms
J_2	2	20ms	6ms	70ms	?	2ms
J_3	3	10ms	2ms	12ms	2ms	0ms

CS	Resource	WCET	In Job
$Z_{1,1}$	S_1	2ms	J_1
$Z_{2,1}$	S_1	4ms	J_2

$$R_2^{(1)} = B_2 + C_2 + \sum_{n:P_n > P_2} C_n * \left\lceil \frac{R_2^{(s-1)}}{A_n} \right\rceil = 8\text{ms} + C_3 * \left\lceil \frac{R_2^{(0)}}{A_3} \right\rceil = 8\text{ms} + 2\text{ms} * \left\lceil \frac{8\text{ms}}{12\text{ms}} \right\rceil = 10\text{ms}$$



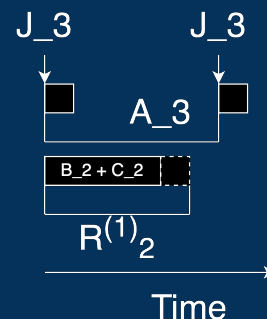
Precise preemption worked example

Job	P	Deadline	WCET	Interarrival	Response Time	B_i
J_1	1	56ms	10ms	100ms	?	0ms
J_2	2	20ms	6ms	70ms	?	2ms
J_3	3	10ms	2ms	12ms	2ms	0ms

CS	Resource	WCET	In Job
$Z_{1,1}$	S_1	2ms	J_1
$Z_{2,1}$	S_1	4ms	J_2

- Just by inspecting the situation graphically, we see that the increased length of $R_2^{(1)}$ doesn't increase the amount of preemptions, we can calculate to be sure

$$R_2^{(2)} = 8\text{ms} + 2\text{ms} * \left\lceil \frac{10\text{ms}}{12\text{ms}} \right\rceil = 8\text{ms} + 2\text{ms} * 1 = 10\text{ms} = R_2^{(1)}$$



Precise preemption worked example

Job	P	Deadline	WCET	Interarrival	Response Time	B_i
J_1	1	56ms	10ms	100ms	?	0ms
J_2	2	20ms	6ms	70ms	10ms	2ms
J_3	3	10ms	2ms	12ms	2ms	0ms

CS	Resource	WCET	In Job
$Z_{1,1}$	S_1	2ms	J_1
$Z_{2,1}$	S_1	4ms	J_2

- We've obtained a tighter bound on R_2 than in the overapproximation case, 10ms instead of 12ms

Precise preemption worked example

Job	P	Deadline	WCET	Interarrival	Response Time	B_i
J_1	1	56ms	10ms	100ms	?	0ms
J_2	2	20ms	6ms	70ms	10ms	2ms
J_3	3	10ms	2ms	12ms	2ms	0ms

CS	Resource	WCET	In Job
$Z_{1,1}$	S_1	2ms	J_1
$Z_{2,1}$	S_1	4ms	J_2

- Finally, R_1 , again, neither blocking nor the WCET of J_1 are affected by preemptions, we keep the values from before, and apply the recurrence relation

$$R_1 := \begin{cases} R_1^{(0)} = B_1 + C_1 = 0\text{ms} + 10\text{ms} = 10\text{ms} \\ R_1^{(s)} = B_1 + C_1 + \sum_{n:P_n > P_1} C_n * \left\lceil \frac{R_1^{(s-1)}}{A_n} \right\rceil = 10\text{ms} + \sum_{n:P_n > P_1} C_n * \left\lceil \frac{R_1^{(s-1)}}{A_n} \right\rceil \end{cases}$$

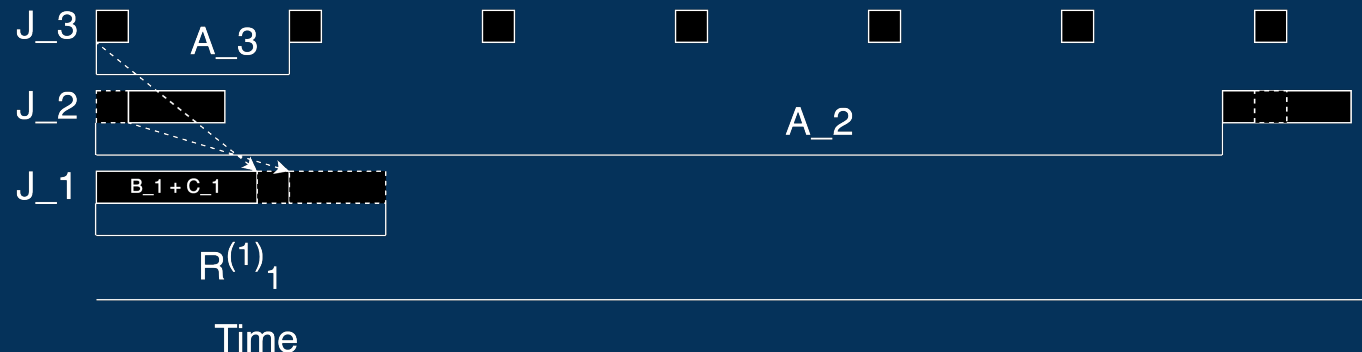
- Notice, we may be preempted by two jobs here since $P_3 > P_1$ and $P_2 > P_1$

Precise preemption worked example

Job	P	Deadline	WCET	Interarrival	Response Time	B_i
J_1	1	56ms	10ms	100ms	?	0ms
J_2	2	20ms	6ms	70ms	10ms	2ms
J_3	3	10ms	2ms	12ms	2ms	0ms

CS	Resource	WCET	In Job
$Z_{1,1}$	S_1	2ms	J_1
$Z_{2,1}$	S_1	4ms	J_2

- $R_1^{(1)} = B_1 + C_1 + C_2 * \left\lceil \frac{R_1^{(0)}}{A_2} \right\rceil + C_3 * \left\lceil \frac{R_1^{(0)}}{A_3} \right\rceil =$
 $= 10\text{ms} + 6\text{ms} * \left\lceil \frac{10\text{ms}}{70\text{ms}} \right\rceil + 2\text{ms} * \left\lceil \frac{10\text{ms}}{12\text{ms}} \right\rceil = 10\text{ms} + 6\text{ms} * 1 + 2\text{ms} * 1 = 18\text{ms}$
- Inspecting, for $R_1^{(2)}$, we will be preempted by J_2 one more time, $\left\lceil \frac{18\text{ms}}{12\text{ms}} \right\rceil > \left\lceil \frac{10\text{ms}}{12\text{ms}} \right\rceil$

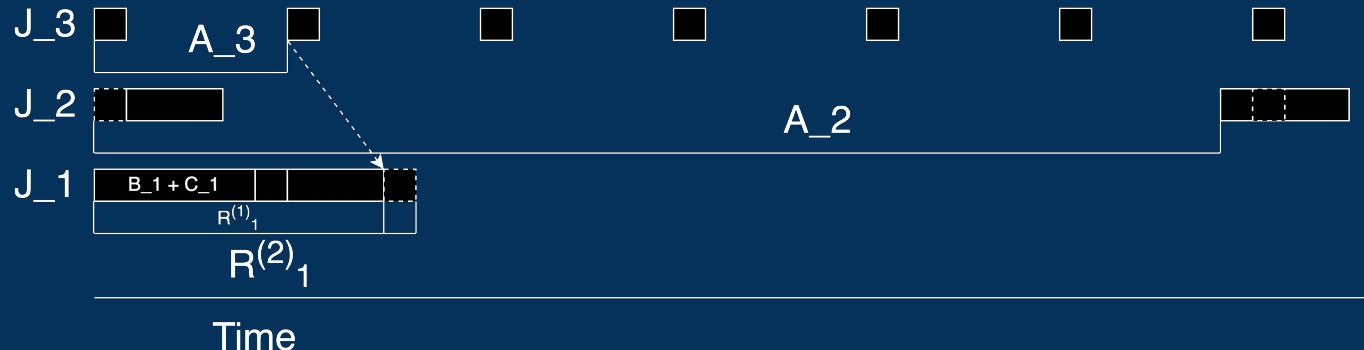


Precise preemption worked example

Job	P	Deadline	WCET	Interarrival	Response Time	B_i
J_1	1	56ms	10ms	100ms	?	0ms
J_2	2	20ms	6ms	70ms	10ms	2ms
J_3	3	10ms	2ms	12ms	2ms	0ms

CS	Resource	WCET	In Job
$Z_{1,1}$	S_1	2ms	J_1
$Z_{2,1}$	S_1	4ms	J_2

- $R_1^{(2)} = B_1 + C_1 + C_2 * \left\lceil \frac{R_1^{(2)}}{A_2} \right\rceil + C_3 * \left\lceil \frac{R_1^{(2)}}{A_3} \right\rceil =$
 $= 10\text{ms} + 6\text{ms} * \left\lceil \frac{18\text{ms}}{70\text{ms}} \right\rceil + 2\text{ms} * \left\lceil \frac{18\text{ms}}{12\text{ms}} \right\rceil = 10\text{ms} + 6\text{ms} * 1 + 2\text{ms} * 2 = 20\text{ms}$
- Inspecting, for $R_1^{(3)}$, we shouldn't run into more preemptions, we can count to check



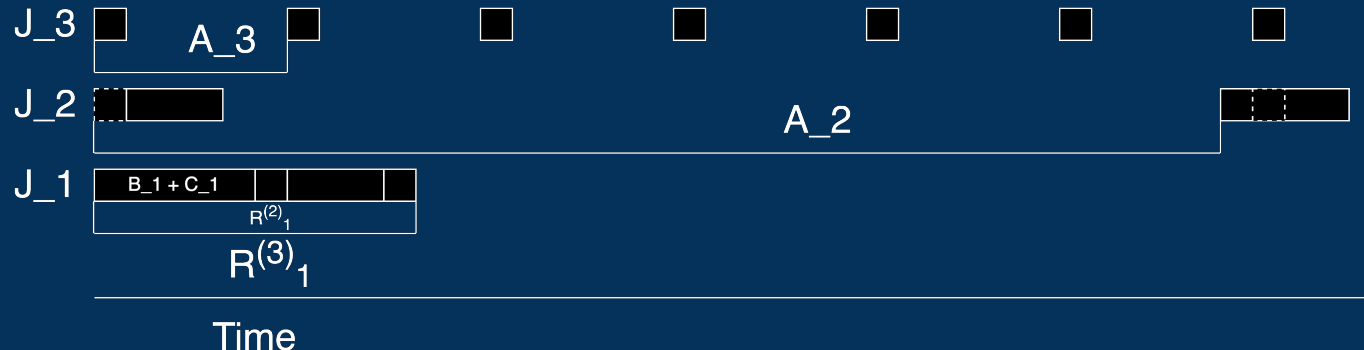
Precise preemption worked example

Job	P	Deadline	WCET	Interarrival	Response Time	B_i
J_1	1	56ms	10ms	100ms	20ms	0ms
J_2	2	20ms	6ms	70ms	10ms	2ms
J_3	3	10ms	2ms	12ms	2ms	0ms

CS	Resource	WCET	In Job
$Z_{1,1}$	S_1	2ms	J_1
$Z_{2,1}$	S_1	4ms	J_2

- $$R_1^{(3)} = B_1 + C_1 + C_2 * \left\lceil \frac{R_1^{(3)}}{A_2} \right\rceil + C_3 * \left\lceil \frac{R_1^{(3)}}{A_3} \right\rceil =$$

$$= 10\text{ms} + 6\text{ms} * \left\lceil \frac{20\text{ms}}{70\text{ms}} \right\rceil + 2\text{ms} * \left\lceil \frac{20\text{ms}}{12\text{ms}} \right\rceil = 10\text{ms} + 6\text{ms} * 1 + 2\text{ms} * 2 = 20\text{ms}$$
- Indeed, we converge around $R_1 = 20\text{ms}$



Schedulability criterion under SRP

- By applying a more precise preemption model, we've been able to obtain tighter bounds on response times
 - Notice here, in both cases the bound is **safe**, i.e. given our parameter assumptions (WCET, interarrival time) are correct, e.g. R_2 will never be **larger** than 12ms or 10ms
- We can define whether our system is schedulable as follows:
 - For all jobs J_i , $R_i > D_i$
 - Intuitive, otherwise we are missing deadlines
 - Defining the load factor L_a of a job J_a as $\frac{A_a}{C_a}$
 - The ratio of the arrival period we need to spend executing a job
 - For all jobs J_i , $\sum(L_i) < 1$
 - This also makes intuitive sense, otherwise we are trying to do more work than available time on the processor
- Given these two properties, we know our system fulfills all deadlines in perpetuity

SRP in research

- Recall, SRP was originally defined for Earliest Deadline First, RTIC uses static priorities
- **Theoretically**, EDF is a better scheduling algorithm than static priority scheduling
 - Any set of jobs that is inherently schedulable (schedulable by some algorithm) is schedulable under EDF
 - This is not the case for static priority scheduling
 - Scheduler complexity throws a wrench in the works, however
- Last month, paper submitted on interrupt controller which allows hardware accelerated, zero-overhead EDF scheduling
 - Optimal solution, but, requires custom hardware
- Currently working on optimal implementation of SRP-EDF under commonplace NVIC
 - Scheduler still has some software cost, whether this pans out being better than static priority SRP is a pending question

Wrap-up

- Today, we've covered how we can ensure real-time properties of our system are upheld
 - Adherence to deadlines
- Doing this on paper is tedious, easily automatable
 - In the lab, you will implement an analysis tool that derives schedulability of a system given job parameters

Todo now!

- Finish up lab 3, final deadline is tonight
- Lab 4 is now updated and ready for you to work with
 - Fork the repo and start poking
 - If issues arise, Discord or lab session

Todo until next session

- Session on Wednesday is lab session, initial Lab 4 deadline is Thursday
- Next lecture is **Monday**
 - Shorter session, practicalities on grading, especially for higher grades
 - Some examples of what you may do as home exam

